Exercices de Compléments d'algorithmique

Série 8, 9 et 10 – Akinator

(10 avril 2025)

Département d'Informatique – Faculté des Sciences – UMONS

Pré-requis : Cours jusqu'au **Chapitre 8**.

Objectifs : Combiner les connaissances du cours pour créer un projet de petite taille.

---

1 Akinator, le Génie du web

En 2007, un certain Arnaud Megret décide, pour s'amuser, de créer une intelligence artificielle qui pose des questions à un joueur pour deviner à quoi ou qui (objet, personne, personnage, animal, ...) il pense. Il personnifie cette intelligence artificielle en l'appelant Akinator, et en lui donnant une apparence de génie. Vous pouvez l'admirer ci-dessous.



A la base, Akinator n'est pas très malin. Il ne connait que 100 personnages et 100 questions. Ainsi, il n'est initialement pas capable de deviner souvent ce que le joueur pense. L'idée pour l'améliorer est qu'il enregistre les informations de chaque partie pour renforcer sa base de connaissance. Ainsi, après chaque partie où il perd, il demande à l'utilisateur à quel personnage il pensait.

Si vous ne l'avez jamais testé, le jeu est accessible en cliquant sur ce lien. Vous pouvez faire une partie ou deux pour comprendre le style de jeu. Aujourd'hui, Akinator ne vous semblera probablement

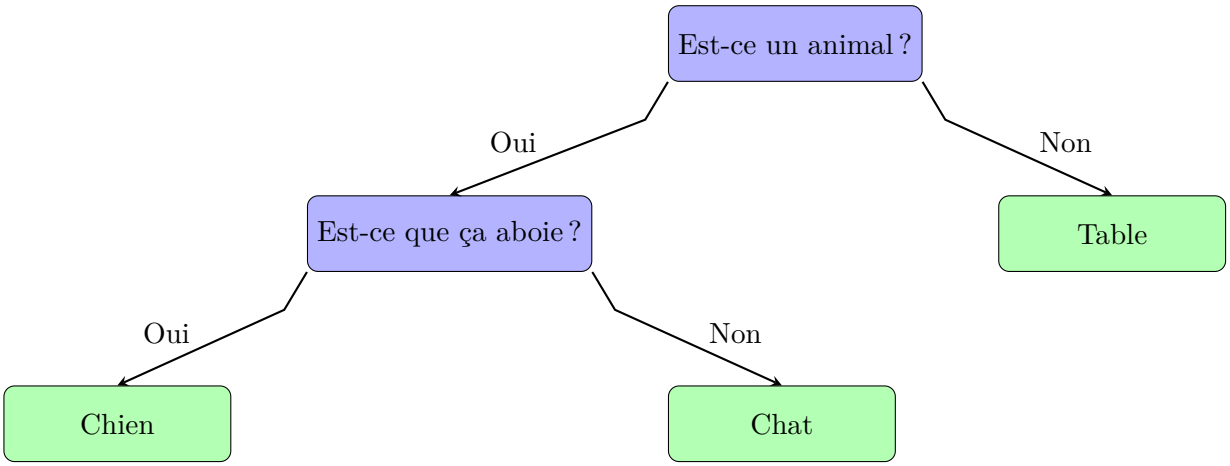
pas si impressionnant étant donné les progrès récents en intelligence artificielle, mais à l'époque le jeu était très célèbre.

Dans cette séance, et dans les deux suivantes, nous allons réaliser une application qui tente d'appliquer le même principe (à échelle bien plus réduite tout de même, vous aurez l'occasion d'en apprendre plus sur l'intelligence artificielle plus tard dans votre cursus).

2 La mémoire d'Akinator

Nous nous contentons d'un fonctionnement bien plus simple que celui décrit plus haut. Notre Akinator pose des questions simples où l'on répond juste oui ou non. Pour ce faire, il suit un arbre de décision tel que celui ci-dessous. Il commence par poser la première question, puis, selon la réponse, progresse dans l'arbre. A chaque fois qu'il progresse dans l'arbre, il peut tomber soit sur une autre question (en bleu dans l'arbre), soit sur une réponse (en vert dans l'arbre).

Notez que cette façon de faire ne sera clairement pas optimale. Dans notre exemple, il faut poser deux questions pour deviner "chien", alors qu'a priori on pourrait simplement deviner cette réponse en sachant que notre sujet aboie. Ici, c'est assez peu important, mais quand notre base de connaissance augmentera, ça sera un peu plus ennuyant. Malheureusement, par contrainte de temps, nous accepterons simplement ce problème, même s'il existe de meilleures façons de travailler.



Pour votre information, un tel arbre est un arbre binaire. En effet, cet arbre est composé de nœuds qui ont chacun au plus (ici, exactement) deux fils.

Nous considérerons plus tard comment mettre à jour cet arbre. Pour l'instant, nous nous contentons de réfléchir à comment le stocker. A l'instar des listes chaînées, chaque question aura une référence vers ce qui suit une réponse positive, et une référence vers ce qui suit une réponse négative. Comme on ne sait pas encore s'il s'agit d'une question ou d'une réponse, il va falloir utiliser le concept d'héritage. En tout cas, nos objets ne devront pas être très complexes, on s'attend juste à ce qu'ils soient capables de renvoyer soit leur réponse, soit leur question.

1 Créez une classe abstraite Node qui a juste une méthode abstraite guess.

2 Créez une classe Answer qui hérite de Node. Cette classe stocke juste une chaîne de caractères qui est la réponse devinée par notre Akinator quand on atteint ce nœud. La méthode guess renvoie simplement cette valeur. N'oubliez pas votre constructeur.

3 Créez une classe Question qui hérite de Node. Cette classe contient une question, une référence vers le nœud pour une réponse positive et une référence vers le nœud pour une réponse négative. Comprenez bien qu'un nœud peut être soit une réponse, soit une autre question ! La méthode guess doit renvoyer un triplet contenant la question et chacune de références vers ses sous-nœuds. N'oubliez pas votre constructeur.

3 Akinator devine

Maintenant que nous pouvons stocker notre base de connaissances, nous pouvons créer un Akinator qui l'utilise pour tenter de deviner.

4 Créez une classe Akinator. Pour l'instant, cette classe contient juste un nœud que l'on appellera "root" car c'est la racine de notre arbre de décision. N'oubliez pas le constructeur.

5 Instanciez des objets Question et Answer pour avoir une base de connaissances de base. N'oubliez pas de bien ajouter à chaque question deux nœuds (qui peuvent être d'autres questions ou des réponses ... N'hésitez pas à faire preuve de créativité !). Instanciez un objet Akinator qui prend en racine la première question.

Pour les deux exercices précédents, si vous n'êtes pas certain de votre implémentation, appelez un assistant avant de passer à la suite.

6 Implémentez maintenant une méthode play dans la classe Akinator. Cette méthode doit parcourir l'arbre de la manière décrite dans cet énoncé. Pour rappel, on commence par poser la question à la racine, et selon la réponse de l'utilisateur, on explore l'un des deux sous-nœuds. On continue ainsi jusqu'à atteindre une réponse finale, qu'on affiche alors à l'écran.

Attention à votre implémentation. L'affichage des questions, la récupération de la réponse de l'utilisateur, et tout le travail de logique doit se faire dans la classe Akinator. A priori, il n'est pas nécessaire de recommencer à la racine à chaque réponse de l'utilisateur.

Pour cet exercice et pour tous les suivants, pensez à bien contrôler et simplifier les inputs de l'utilisateur. Par exemple, s'il rentre " Oui.", ça devrait être considéré comme "oui".

4 Akinator apprend

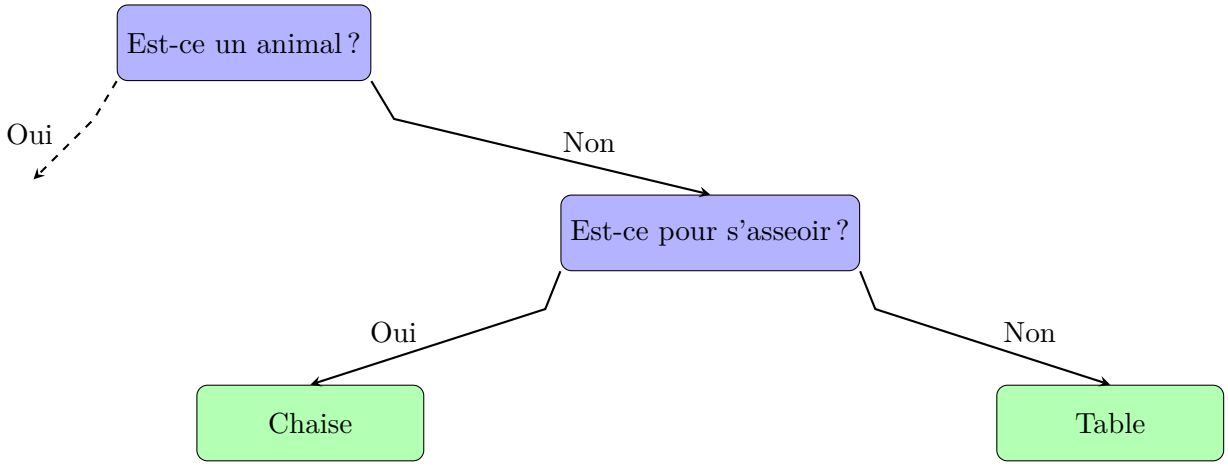
Le problème maintenant, c'est que notre Akinator ne deviendra jamais très compétent en se basant juste sur les données qu'on lui a fournies. Il faut qu'il soit capable d'apprendre de chacune de ses défaites. Nous allons donc vérifier s'il répond correctement ou non, et demander à l'utilisateur d'aider Akinator à s'améliorer.

7 Commencez par modifier la méthode play pour que celle-ci vérifie si la réponse proposée par Akinator était correcte ou non. Faites le en demandant à l'utiliateur si c'est le cas, puis affichez un message de victoire ou de défaite de la part d'Akinator.

8 Demandez à l'utilisateur d'entrer une question qui permet de distinguer la réponse proposée par Akinator de celle que l'utilisateur avait en tête. Par exemple, si la réponse proposée par Akinator est "table" alors que l'utilisateur pensait à "chaise", l'utilisateur pourrait entrer "Est-ce que c'est fait pour s'asseoir dessus ?". Pensez à demander à l'utilisateur également quelle réponse correspond à oui ou à non.

9 Enfin, en utilisant les informations précédentes, mettez à jour notre arbre. Pour ce faire, le nœud qui était précédemment une Answer doit devenir une Question (celle fournie par l'utilisateur), qui pointe vers deux Answer (l'une contenant la réponse originale, l'autre contenant celle à laquelle pensait l'utilisateur).

Attention ! Il ne suffit pas de créer ce nœud. Il faut le mettre en tant que sous-nœud du nœud précédent. Dans l'exemple que je viens de donner, le nœud demandant "Est-ce un animal" devrait pointer vers ce nouveau nœud dans le cas d'une réponse négative. Cet exemple est illustré ci-dessous. N'hésitez pas à comparer l'exemple précédent à ce nouvel exemple pour voir ce qui change exactement.



Notez de nouveau que cette façon de faire n'est pas optimale. Si l'on veut insérer une nouvelle réponse, on est obligés de le faire tout en bas de l'arbre. Notre arbre pourrait aussi être fortement déséquilibré (par exemple, si l'on répond toujours non et qu'on ajoute donc toujours à droite), ce qui le rendrait assez peu efficace.

5 Akinator n'oublie pas

Pour l'instant, à chaque fois que l'on ferme notre application, Akinator retourne aux données que l'on a initialement inscrites dans l'application. Il apprend, mais il oublie rapidement. Il nous faut donc une solution pour pouvoir inscrire ces données quelque part de manière plus permanente afin de pouvoir les récupérer plus tard. Il existe de nombreuses solutions pour faire ça. Dans le cadre de ce projet, nous allons simplement sauvegarder nos données dans un fichier, et recharger ce fichier à chaque lancement du programme. Plus précisément, nous allons utiliser le format JSON (JavaScript Object Notation). C'est un format de données qui est intéressant de par sa facilité d'utilisation et de lecture (même pour un humain).

Vous pouvez voir ci-dessous un premier exemple qui enregistre les informations d'une personne liée à l'UMONS. Vous noterez que l'on peut stocker des chaînes de caractère, des nombres, des booléens, des objets ou d'autres choses encore.

```
{
  "nom": "Valentin Dusollier",
  "ville": "Mons",
  "estDoctorant": true,
  "cours": ["Algorithmique et programmation imperative", "Complements
     d'algorithmique"],
  "contact": {
    "email": "valentin.dusollier@umons.ac.be",
    "telephone": "+33 333333 3333"
  }
}
```

Si nous reprenons notre exemple d'arbre de décision de la série précédente, nous pourrions obtenir un fichier JSON tel que ci-dessous. Le format est très simple.

Les objets Question mènent à quatre entrées :

— "type" : "Question"

— "prompt" : "Question qui permet de distinguer"

— "yes_node" : { ... } qui contient les informations d'un autre nœud (une Question ou une Answer), celui qui correspond à une réponse positive.
— "no_node" : { ... } qui contient les informations d'un autre nœud (une Question ou une Answer), celui qui correspond à une réponse négative.

Les objets Answer mènent à deux entrées :
— "type" : "Answer"
— "answer" : "Réponse donnée par Akinator"

```json
{
    "type": "Question",
    "prompt": "Est-ce un animal ?",
    "yes_node": {
        "type": "Question",
        "prompt": "Est-ce que \u00e7a aboie",
        "yes_node": {
            "type": "Answer",
            "answer": "Chien"
        },
        "no_node": {
            "type": "Answer",
            "answer": "Chat"
        }
    },
    "no_node": {
        "type": "Answer",
        "answer": "Voiture"
    }
}
```

En Python, la librairie json permet de générer des fichiers JSON. Pour l'instant, une seule fonction nous intéresse dans cette librairie, c'est la fonction dump. Nous vous invitons à aller lire sa documentation par vous-même. La chose essentielle à comprendre pour le moment, c'est qu'elle prend un dictionnaire pour l'écrire dans un fichier JSON. Il est donc nécessaire de pouvoir transformer notre arbre en dictionnaire.

10 Ajoutez à votre classe abstraite une méthode to_dict, puis implémentez la dans vos classes Answer et Question. Cette méthode, comme son nom l'indique, doit renvoyer un dictionnaire. Utilisez l'exemple de fichier JSON ci-dessus pour savoir comment ces dictionnaires sont structurés. Par exemple, '"type" : "Question"' est une entrée du dictionnaire où 'type' est la clé et 'Question' est la valeur associée. Pour vous aider, nous vous précisons que cette fonction aura besoin d'être récursive.

11 Ajoutez à votre classe Akinator une méthode save_to_json. Cette méthode prend en paramètre un nom de fichier, et utilise la fonction json.dump pour écrire un fichier JSON basé sur l'arbre contenu dans l'instance d'Akinator.

12 Modifiez votre code pour qu'à chaque fois que l'arbre est modifié (après une partie), il soit sauvegardé dans un fichier JSON.

Prenez le temps de faire une partie et de vérifier que votre fichier JSON s'est bien écrit et est à jour. Pour votre information, on emploie en général le terme de sérialisation pour décrire l'opération que l'on vient de réaliser. Il s'agit de l'encodage d'une information en une suite d'informations plus petites.

De la même manière, nous allons maintenant nous intéresser à la désérialisation. Il ne nous reste en effet plus qu'à être capable de lire ce fichier au lancement de notre programme pour pouvoir récupérer ces informations.

13 Ecrivez une fonction dict_to_node qui prend en paramètres un dictionnaire, et qui renvoie un objet Answer ou Question selon la valeur correspondant à la clé "type". Encore une fois, cette fonction aura besoin d'être récursive.

14 Modifiez votre programme pour qu'il essaye de charger le fichier JSON à son lancement. Si un problème advient (structure JSON non attendue, fichier non existant, ...) prévoyez un Akinator par défaut (vous pouvez garder celui qu'on initialisait jusqu'à maintenant).

6 Akinator pose

Maintenant que notre application est en grande partie fonctionnelle, nous pouvons nous intéresser à sa présentation. Plutôt que de passer par un terminal, nous allons créer une interface graphique.

Dans les exercices qui suivent, les éléments graphiques sont simplement des suggestions. L'important est que la logique reste la même. N'hésitez pas à faire preuve de créativité dans votre présentation.

Faites attention tout de même à l'expérience de l'utilisateur. Il faut que quelqu'un qui ne connait pas votre application puisse facilement l'utiliser. Cela passe par des messages d'avertissement clairs, une interface facilement navigable et encore d'autres choses. Par exemple, quand l'utilisateur a fait deviner correctement un mot à Akinator, ne le ramenez pas immédiatement au menu principal, confirmez lui d'abord que tout s'est bien passé.

Faites un effort à ce niveau, ça vous sera très utile pour de futurs projets. En effet, c'est souvent plus l'ergonomie de l'interface qui est jugée que sa beauté.

Vous pouvez trouver, en dessous des exercices, des exemples très basiques de ce qui est attendu.

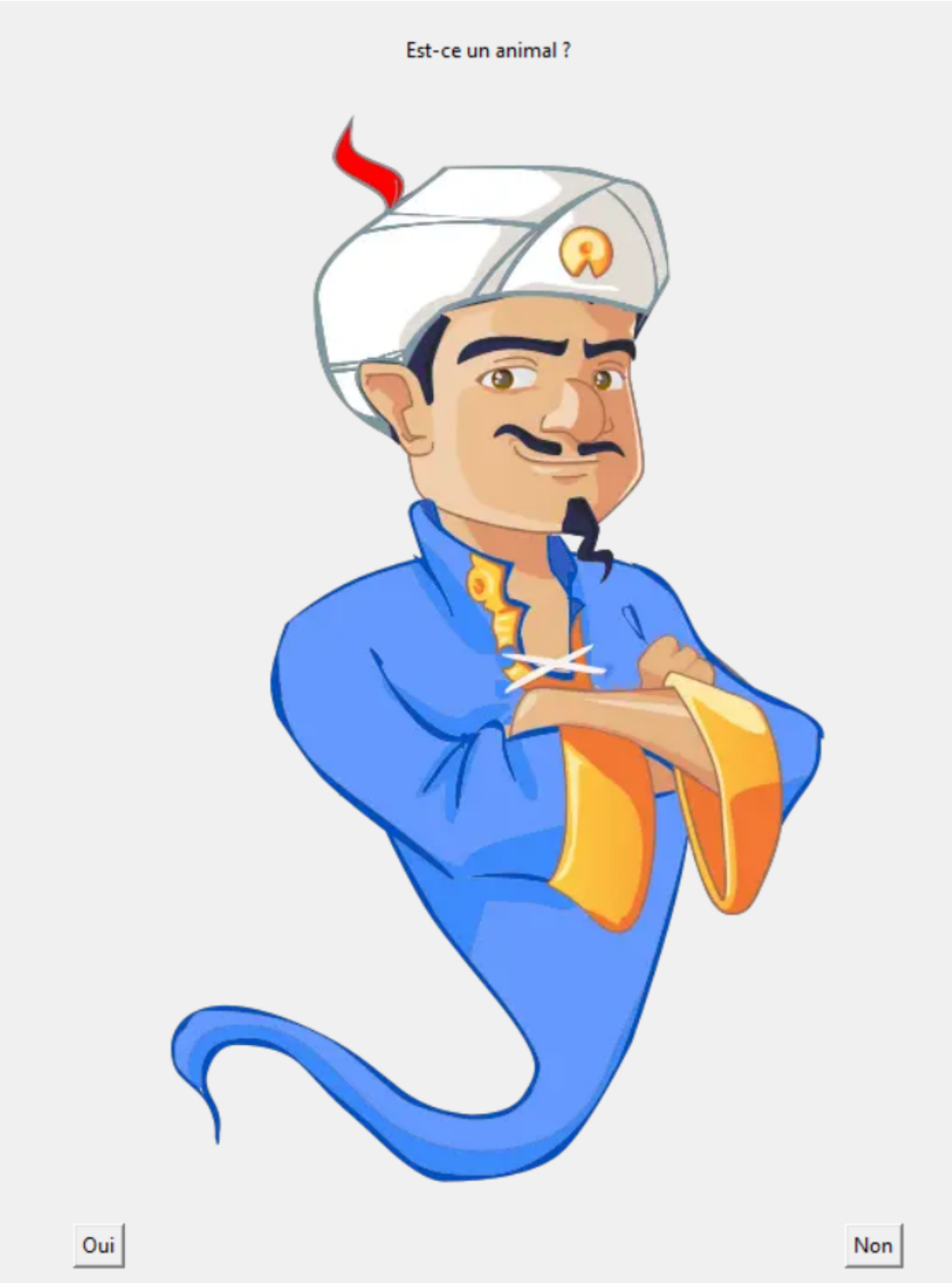
15 Commencez par créer une nouvelle classe AkinatorApp. Nous commençons par créer un simple menu principal. Dans ce menu, on retrouve un bouton "Play", qui lance le jeu (pour l'instant, contentez vous de lancer le jeu dans le terminal), et un bouton "Quit" qui ferme juste l'application.

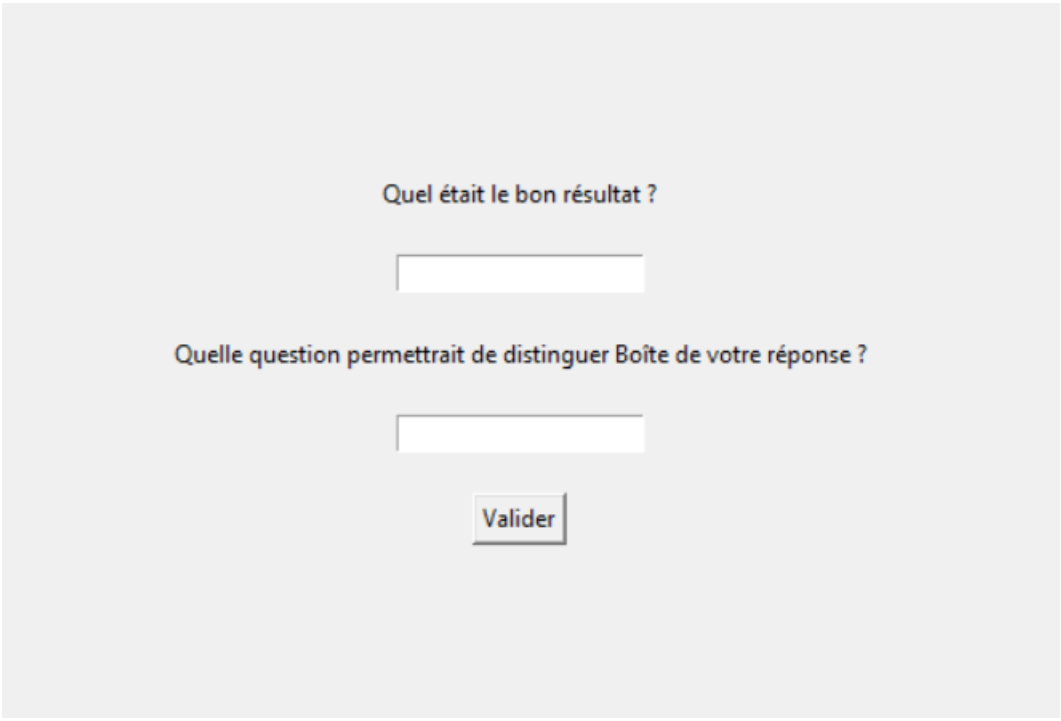
16 Modifiez cette classe pour qu'elle affiche une fenêtre de jeu. N'essayez pas directement de programmer des fonctionnalité, commencez par simplement créer la fenêtre et y mettre les éléments dont nous aurons besoin. Voici quelques suggestions. Il faudrait probablement au minimum un espace où afficher les questions et deux boutons pour que l'utilisateur puisse répondre oui ou non. Pour rendre l'interface plus intéressante, n'hésitez pas à inclure une image représentant Akinator.

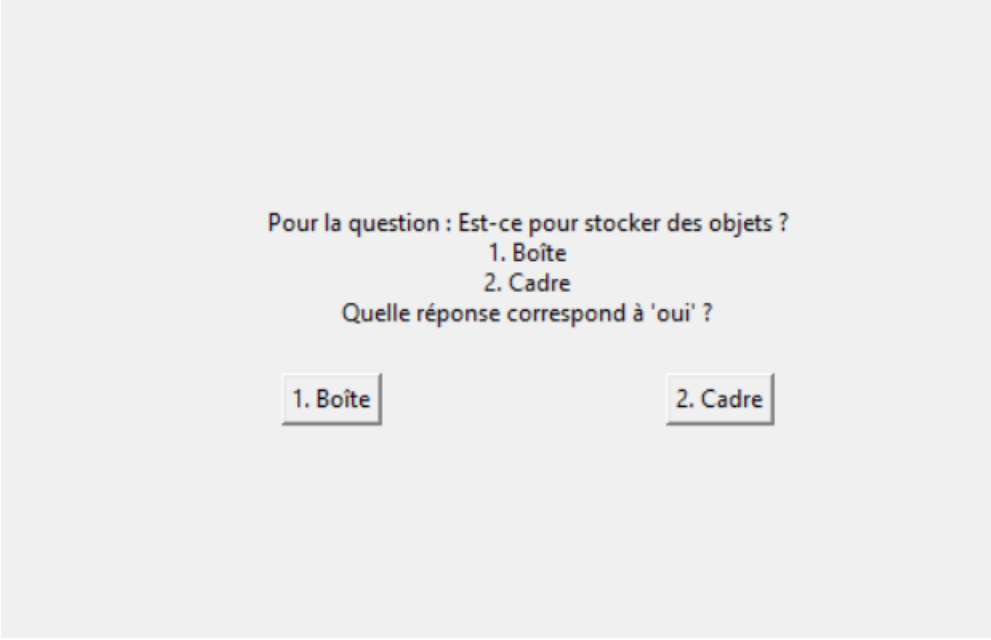
17 Changez cette fenêtre pour qu'elle fasse fonctionner le jeu, sans vous soucier de la fin de partie pour l'instant. Quand le jeu se termine, passez encore par le terminal pour le moment. L'idée est que tout ce qui contrôle la logique du jeu et l'affichage doit être dans la classe AkinatorApp, alors que la classe Akinator sert uniquement à stocker et manipuler les données de l'arbre. Cela va probablement nécessiter de réduire la méthode play de Akinator et de tranférer une partie de sa logique vers AkinatorApp.

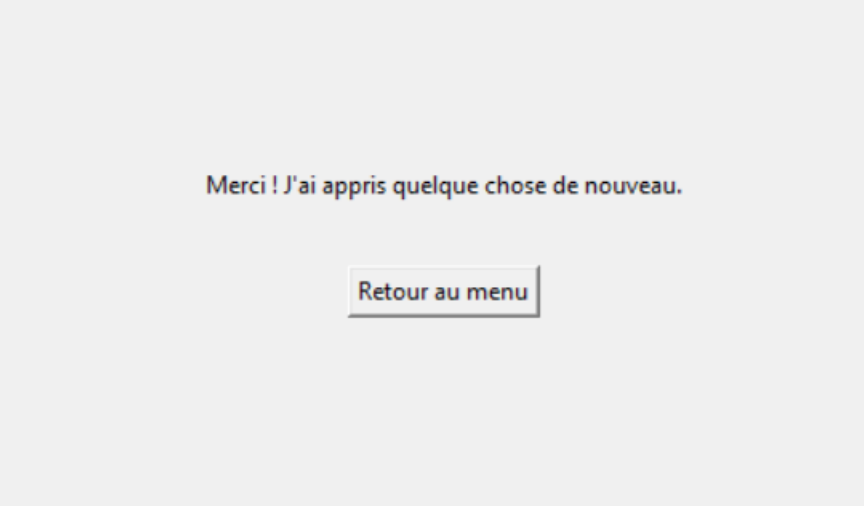
18 Enfin, ajoutez à AkinatorApp la gestion de la fin de la partie. Gardez toujours la même logique.


Play
Quit


Est-ce un animal ?
Oui
Non


Quel était le bon résultat ?
Quelle question permettrait de distinguer Boîte de votre réponse ?
Valider

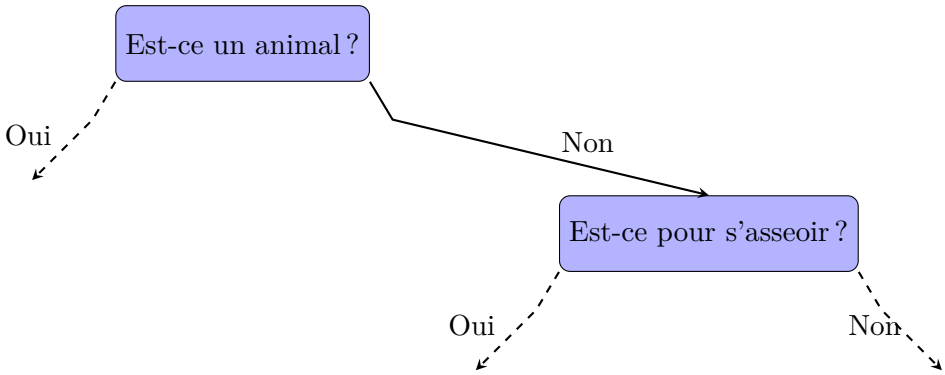

Pour la question : Est-ce pour stocker des objets ?
1. Boîte
2. Cadre
Quelle réponse correspond à 'oui' ?
1. Boîte
2. Cadre



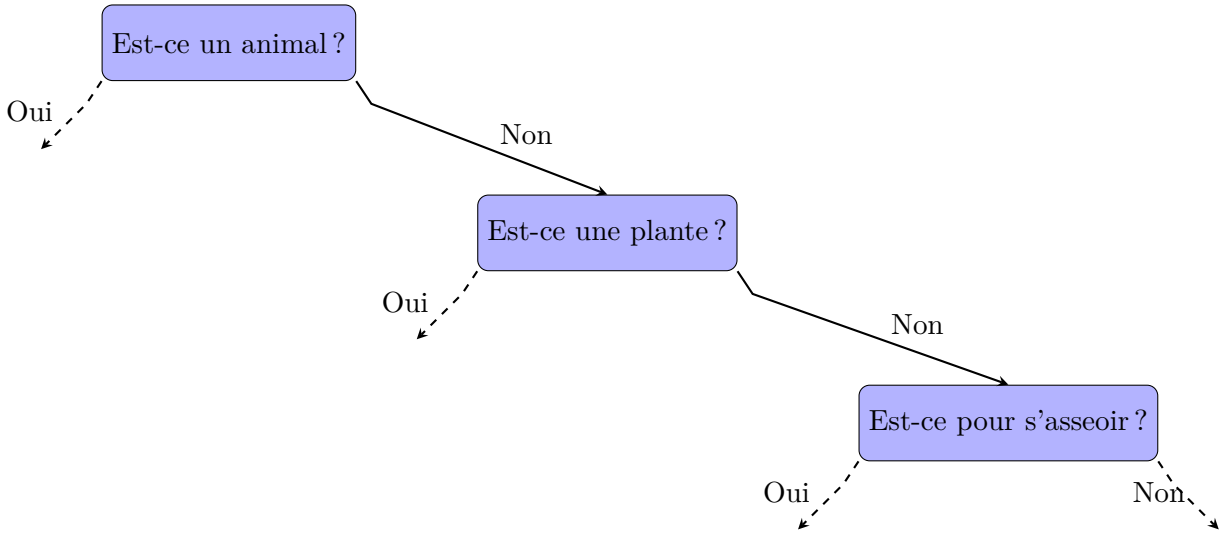
7 En préparation d'un meilleur apprentissage

Un des problèmes avec notre procédé pour améliorer Akinator est que l'on ajoute constamment des nœuds à la fin de l'arbre. Malheureusement, il va être difficile d'ajouter des connaissances différemment en jouant dans le cadre de ce projet (cela demanderait de revoir notre programme de manière général). Cependant, nous pouvons au moins nous préparer à changer notre structure de données d'autres manières.

Reprenons un exemple précédent, après lequel Akinator aurait continué d'apprendre. Notre arbre ressemblerait à la figure ci-dessous. C'est à dire qu'après avoir répondu "non" à "Est-ce un animal ?", on pose la question "Est-ce pour s'asseoir ?". Après cette question, que l'on réponde oui ou non, il se passe forcément autre chose, mais cela ne nous intéresse pas pour notre réflexion actuelle.



Imaginons maintenant que, au lieu de continuer à parcourir notre arbre jusqu'à atteindre une réponse, on souhaite l'éditer pour insérer une question. Ainsi, on obtiendrait la figure suivante où après avoir répondu "Non" à "Est-ce un animal ?", on poserait d'abord la question "Est-ce une plante ?". Alors seulement si on répond non à cette question en vient-on à poser la question "Est-ce pour s'asseoir ?".



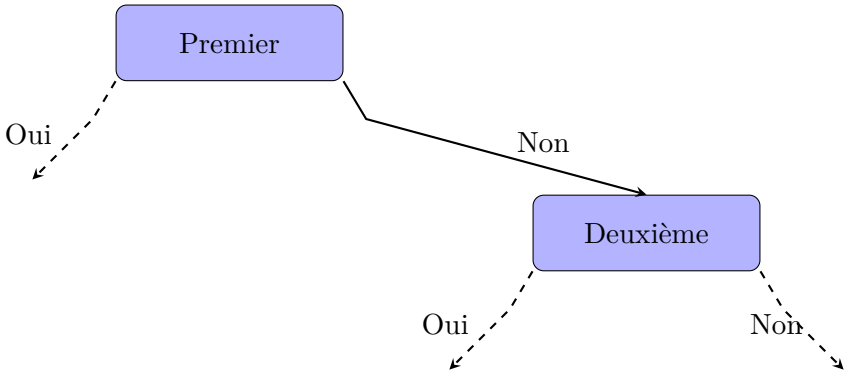
Notez que cette manière de fonctionner n'est pas idéale. Elle peut mener à des arbres incohérents si la réponse à la question que l'on insère n'est pas valide pour les éléments du sous-arbre ainsi créé, ce qui peut être le cas puisqu'on ne le vérifie en aucun cas. Malheureusement, notre structure ne permet pas de le faire facilement et il faudrait revoir notre structure de données entièrement pour pouvoir implémenter ce genre de fonctionnalité. Nous nous contenterons donc de cette manière de faire dans le cadre de ce projet.

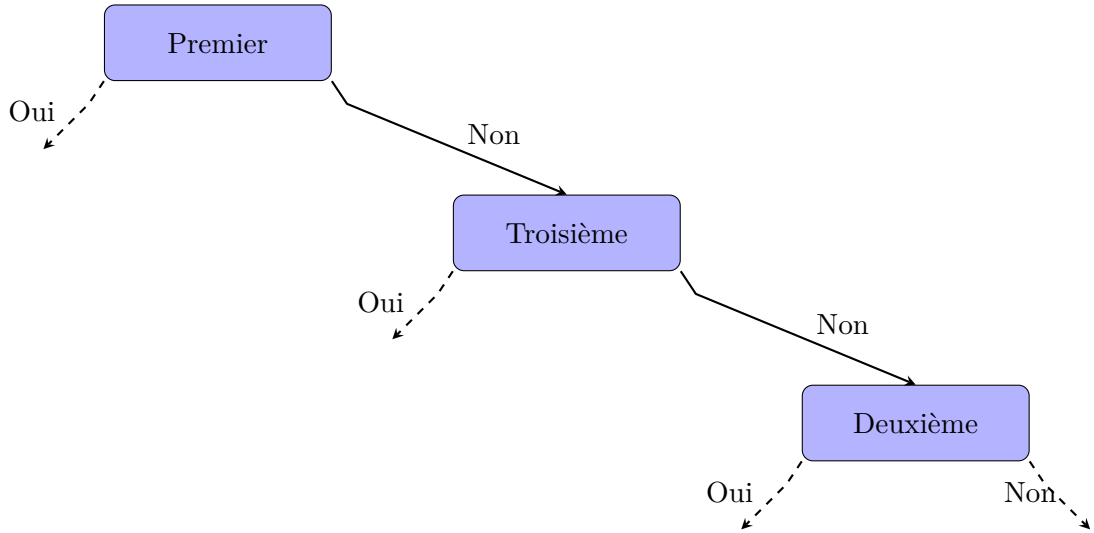
19 Ecrivez une fonction qui prend quatre paramètres au total. Les deux premiers sont les nœuds entre lesquels on vient insérer une nouvelle question (donc, dans notre exemple, "Est-ce un animal ?" et "Est-ce pour s'asseoir ?). Le troisième est le nœud que l'on vient insérer entre les deux premiers. Dans notre exemple, c'est la question "Est-ce une plante ?". Notez bien que dans cette fonction, les paramètres doivent être des nœuds (et non pas forcément des réponses ou des questions). Enfin, le quatrième est un booléen qui indique si le deuxième nœud est le fils droit du troisième nœud (sinon, il s'agit de son fils gauche).

Cette fonction doit réaliser le changement décrit précédemment, c'est à dire venir insérer la nouvelle question entre deux.

Si vous avez du mal à vous y retrouver avec l'explication précédente, n'hésitez pas à vous référer aux deux figures suivantes (qui sont une généralisation de l'exemple du début de l'énoncé). Dans ce cas, le booléen nous indique que le deuxième nœud est effectivement le fils droit du troisième nœud.

Attention, n'oubliez pas de vérifier si le deuxième nœud est le fils gauche ou fils droit du premier nœud.





Testez d'abord cette nouvelle fonction avant d'essayer de réaliser l'exercice suivant, car le fonctionnement sera très similaire.

20 Dans votre application, lorsqu'Akinator est en train de poser une question à l'utilisateur, une troisième option doit maintenant être disponible : "Insérer une question". Cette option demande à l'utilisateur une nouvelle question et une nouvelle réponse. L'application demande alors si la nouvelle réponse correspond à un "oui" ou à un "non".

Par rapport à l'exemple précédent, la question actuelle correspond à "Deuxième" et la question précédente correspond à "Premier". Enfin, la nouvelle question correspond à "Troisième". La nouvelle réponse est alors simplement le deuxième enfant de "Troisième".

8 Fin de projet

Nous pourrions encore implémenter d'autres fonctionnalités mais nous avons déjà bien travaillé. Plutôt que d'essayer de coder encore plus de nouvelles choses, nous vous invitons plutôt à un moment de réflexion par rapport à votre projet actuel.

Essayez de le faire sérieusement, vous aurez à réaliser des projets d'encore plus grande taille dans le futur. Vous aurez d'ailleurs à analyser par vous-même les besoins des applications que vous devrez créer dans le futur, vous ne serez pas guidés dans leur création comme vous l'avez été durant ce TP. Ce n'est pas sans raison que le génie logiciel est un pan de la recherche en sciences informatiques !

Considérez les questions suivantes, et n'hésitez pas à prendre même note de vos réponses si vous avez l'impression qu'elles pourraient vous aider dans vos futurs projets. Notez que c'est un échantillon du genre de questions que vous devrez vous poser lorsque vous rédigerez un rapport pour vos projets dans le futur.

- Quels sont les points forts et les points faibles de votre projet ?
  - Existe-t-il des fonctionnalités que vous auriez aimé implémenter dans ce projet ? Nécessitent-elles des changements fondamentaux par rapport aux projets ? Essayez d'imaginer comment vous feriez.
  - Que devriez-vous faire pour corriger les points faibles auxquels vous avez pensé ?
- A quel point votre code est-il maintenable et/ou lisible ?
  - Si vous devez rajouter une nouvelle fonctionnalité, à quel point serait-ce complexe ?
  - Si vous deviez revenir sur ce projet dans un an, à quel point serait-il difficile de comprendre ce que vous avez fait ?
  - Si vous deviez donner ce code à quelqu'un de votre classe (sans qu'il ait fait le TP), pensez-vous qu'il pourrait le comprendre facilement ?
  - Que pourriez-vous changer dans votre projet pour pouvoir répondre "oui" aux questions précédentes ?

— Que pensez-vous de la testabilité de ce projet ? Si elle est insuffisante, que pourrait-on faire pour l'améliorer ?
— Est-ce que vous avez l'impression que le travail de programmation que vous avez effectué pour ce projet aurait été facilement parallélisable (si oui, grâce à quoi) ? Si vous deviez refaire le projet à plusieurs, comment sépareriez-vous les tâches ?